Applied Cryptography Secure Exchange

ICS 344 Information Security

Faisal Alsalem

Student ID 202245540

1 Design and threat model

Channel: ciphertext + public DH values + salt + IV + HMAC + signature

Trust setup: Bob public DH key to Alice; Alice public signing key to Bob (out of band)

Northbridge Analytics needs to send sensitive result files between Alice and Bob over an insecure channel. The Python program protects a file at Alice's side, saves the protected message as JSON, and checks it at Bob's side before recovering the original bytes. It uses AES-CBC and AES-CTR for encryption, HMAC for integrity, Diffie-Hellman for shared key establishment, and RSA signatures to identify the sender. The recorded run passed all 25 automated tests and six checks of the command-line workflow.

Eve is assumed to be able to read and save everything sent across the channel. Mallory has the same access and can also edit, replace, inject, remove or replay messages. Neither attacker initially has Alice's or Bob's private keys, and neither controls the trusted public-key files. The design does not protect a computer that is already compromised. It also does not hide message sizes or stop an attacker from preventing delivery.

Before any files are sent, Alice receives a trusted copy of Bob's DH public key, and Bob receives a trusted copy of Alice's RSA public key. This setup is important because a public key supplied by Mallory would not establish the other person's identity. The demonstration keeps the keys in one local directory to simulate both people on one computer. The README explains which files each person would need in separate directories.

Alice generates a fresh DH private key, derives encryption and MAC keys, encrypts the file, calculates an HMAC, and signs the protected message. Bob checks the signature, derives the keys, verifies the HMAC and then decrypts. The channel carries the ciphertext, IV or counter start, HKDF salt, mode, protocol label, student ID, public DH values, MAC and signature. Plaintext and private keys stay at the endpoints. Public information is still authenticated so that Mallory cannot change the settings used to interpret the ciphertext.

2 Algorithms and implementation

Mechanism	Parameters	Reason for using it
AES-CBC and AES-CTR	256-bit key; 128-bit block and IV or counter start	Encrypt file contents and compare two modes
PKCS7 padding	128-bit block size; CBC only	Make CBC input a whole number of blocks
HMAC-SHA256	Separate 256-bit key; 256-bit tag	Detect changes to the protected message
Diffie-Hellman	RFC 3526 group 14; 2048-bit prime; generator 2	Agree on a shared secret
HKDF-SHA256	32-byte salt; 64-byte output	Derive separate encryption and MAC keys
RSA-PSS	2048-bit key; e=65537; SHA256; MGF1-SHA256; 32-byte salt	Sign the protected message as Alice

All cryptographic operations use the cryptography library. AES, SHA256 and RSA are not implemented from scratch. The prime used for DH is the published group 14 value from RFC 3526. The library generates the private keys and calculates the shared secrets. The application also checks that a received DH value is in the permitted range and subgroup. T22 confirms that the invalid public value 1 is rejected.

For private values a and b, Alice's public value is A=g^a mod p and Bob's is B=g^b mod p. Alice calculates Z=B^a mod p, while Bob calculates Z=A^b mod p. E03 records the two public values and confirms that both sides independently produced the same shared bytes. The evidence stores SHA256 fingerprints of these results instead of printing the shared secret. Private exponents are not written to the evidence files.

The DH result is passed to HKDF rather than used directly as an AES key. HKDF receives a fresh 32-byte salt and context containing the application label, version, student ID, mode and both DH public values. Its 64-byte output is split into two 32-byte keys. One is used by AES and the other by HMAC. E03 confirms that Alice and Bob obtained matching encryption keys and matching MAC keys. It also confirms that the encryption and MAC keys differed in this run.

The transmitted artifact is a JSON object. Binary fields are Base64 encoded, while DH public values are written in hexadecimal. JSON keys are sorted and unnecessary whitespace is removed before authentication. This gives the sender and receiver the same byte representation to verify. Base64 only makes binary data easier to store in JSON; it does not encrypt it. The full field layout can be inspected in E04 and sample_ctr_envelope.json.

The protected body contains version, label, student_id, mode, alice_dh, bob_dh, salt, iv and ciphertext. HMAC covers the complete body, including the public metadata. Alice then signs the body together with its HMAC. This is Encrypt-then-MAC followed by a signature. Leaving the IV or mode outside authentication would allow an attacker to change how the encrypted data is processed, so those values are included alongside the ciphertext.

Bob first checks the expected fields, protocol label, student ID and recipient public value. He verifies the signature using Alice's trusted public key. He then checks the DH value and field lengths, derives the keys, and verifies the HMAC before decrypting. CBC padding is only removed after authentication succeeds. This order matters because the receiver should not use padding errors to decide whether an unauthenticated message is valid. A failed CLI operation returns a generic rejection.

Private keys are saved as encrypted PKCS8 PEM files using the library's BestAvailableEncryption option. On POSIX systems, the file is created with permissions 0600. The password is requested without echo, although the automated demonstration can provide a temporary password through an environment variable. T24 checks that the private-key file is encrypted and cannot be loaded with the wrong password. No private-key files or passwords are included in the submitted package.

3 Symmetric encryption results

The main inputs are a short message beginning with student ID 202245540 and a newly generated binary file containing 1,031 random bytes. In the recorded run, the short message is 49 bytes long. The binary file was deliberately chosen so that its length is not divisible by AES's 16-byte block size. Empty input and an exactly aligned 16-byte message are also tested. E01 records the environment, run time and random file fingerprint.

Input	Plaintext bytes	CBC ciphertext bytes	CBC padding bytes	CTR ciphertext bytes
Short text	49	64	15	49
Binary file	1031	1040	9	1031
Empty input	0	16	16	0
One full block	16	32	16	16

These lengths show the main difference between the two modes. CBC needs complete blocks, so PKCS7 adds enough bytes to reach the next block boundary. An input that is already aligned still receives a full padding block. This explains why the 16-byte input becomes 32 bytes and the empty input becomes 16 bytes. CTR does not need padding because it XORs the required number of plaintext bytes with a generated keystream. Its ciphertext length therefore matches the plaintext length.

E02 records the ciphertext, tag and IV for both main inputs in each mode. The same logical plaintext is encrypted again for comparison. All four repeated ciphertexts differ, and all four complete protected messages differ. The normal sender generates fresh DH material, salt and IV each time, so this behaviour is expected. Comparing the entire JSON object alone would not be enough to check encryption randomness because the RSA-PSS signature is also randomized. The tests therefore compare ciphertexts separately.

CBC uses a fresh unpredictable 16-byte IV. CTR uses a 16-byte initial counter, followed by consecutive counter blocks. For CTR, the important condition is that a counter stream must not repeat under the same key. Normal transfers also derive fresh keying material for every envelope. The current design is intended for small laboratory files; it does not manage persistent counters for a busy service using long-lived keys.

T01-T08 show that both modes recover the exact input, including empty and block-aligned data. They test correctness, but they do not establish integrity by themselves. T10 checks a modified protected message, and T11 checks the HMAC separately. Both reject the altered ciphertext. Encryption and integrity therefore have separate evidence: correct decryption shows that the data can be recovered, while tamper rejection shows that a changed artifact is not accepted.

4 CTR nonce reuse experiment

The selected misuse is reusing a CTR counter stream. The experiment uses two equal-length messages of 39 bytes and one random AES key. The first plaintext is treated as known to Eve. The second represents the confidential message to be recovered. Keeping the messages equal in length makes it possible to show complete recovery instead of only a shared prefix. E06 stores the inputs and results so that the XOR calculation can be checked directly.

The baseline encrypts the two messages with different initial counters. The misuse changes only the second initial counter so that it equals the first. The key, plaintexts and encryption function stay the same. For the correction, the second initial counter is replaced with a fresh value again. The first ciphertext is retained throughout. This isolates the counter condition instead of changing several settings and then guessing which one caused the result.

CTR encryption has the form C=P XOR S, where S is the generated keystream. If the same key and counter stream are reused, then C1=P1 XOR S and C2=P2 XOR S. XORing the two ciphertexts cancels S, giving C1 XOR C2=P1 XOR P2. Since Eve knows P1, she can calculate P2=C1 XOR C2 XOR P1. The prediction is therefore

exact recovery of the second plaintext in the misuse case.

Configuration	Second counter start	Expected result	Observed result
Baseline	Different from the first	This recovery method fails	No exact recovery
Misuse	Equal to the first	Recover the second plaintext	Exact recovery
Correction	Fresh value again	This recovery method fails	No exact recovery

T18 confirms the baseline, T19 confirms the misuse and T20 confirms the correction. E06 includes hexadecimal ciphertexts, their XOR and the recovered bytes. In the misuse case, both xor_matches_plaintexts and recovered_exact are true. In the other cases, exact recovery is false. These results show the expected confidentiality failure caused by reusing the stream. They do not suggest that the AES primitive itself has been broken.

The attack is passive: Eve reads two legitimate ciphertexts without editing either one. A correct HMAC or signature would therefore not prevent the information leak. They could authenticate both messages while the repeated keystream still exposes their XOR. The correction must prevent stream reuse under the same key. The failed recovery after correction shows that this particular attack no longer works in that run; it is not a proof that the whole application is secure.

5 Comparative security tests

The test suite records the input condition, expected result, actual result and pass or fail status. The full list is in E05 and tests.csv. All 25 tests passed in the recorded run. The separate CLI checks run setup, send and receive as real processes rather than calling only the Python functions. E08 records successful CBC and CTR transfers, rejection after tampering, and protection of an existing output file.

Tests	Condition	Expected result	Actual result	Status
T01-T08	Four input types in both modes	Exact round trips	Exact matches	Pass
T09	Repeat each main input	Different ciphertexts and envelopes	All four differ	Pass
T10	Change a ciphertext bit	Reject envelope	InvalidSignature	Pass
T11	Check changed data with HMAC alone	Reject	InvalidSignature	Pass
T12-T13	Wrong recipient key or MAC key	Reject both	Both rejected	Pass
T14-T15	Original and modified signature input	Accept original; reject change	As expected	Pass
T16	Independent DH and HKDF calculations	Matching secrets and keys	All match	Pass
T17	Wrong signature verification key	Reject	InvalidSignature	Pass
T18-T20	CTR baseline, misuse and correction	Fail, recover, fail	As expected	Pass
T21-T23	Changed IV, invalid DH value, wrong ID	Reject each	All rejected	Pass
T24	Wrong private-key password	Reject key loading	Rejected	Pass
T25	Replace message and unkeyed hash	Hash accepts; HMAC rejects	As expected	Pass

A rejection counts as a passing test when rejection is the expected behaviour. For example, T12 rejects a different Bob receiving key before decryption, while T13 fails HMAC verification with the wrong MAC key. The program does not rely on unreadable plaintext or incorrect CBC padding to detect a wrong key. The CLI

tamper checks also confirm that no new plaintext output file is created after rejection. These checks cover the assignment requirements, but they are not exhaustive testing of every possible malformed input.

6 Applied analysis

E07 gives a simple example of why an unkeyed hash is not a MAC. The original message is pay=100. The attacker replaces it with pay=900 and calculates a fresh SHA256 digest for the replacement. A receiver that compares the message with this attacker-supplied digest accepts it. No hash collision is needed because the attacker changes both the data and the digest. The original HMAC rejects the changed message because a valid replacement tag requires the secret key. T25 records both outcomes.

DH successfully establishes matching keying material in T16, but matching keys alone do not identify the peer. If public values can be replaced without authentication, Mallory can establish separate shared secrets with each person. This implementation depends on trusted public-key provisioning and Alice's signature. Bob's DH public key is trusted before Alice sends anything, and Bob checks Alice's signed envelope with her trusted RSA key. T17 shows that the signature does not verify under a different key. Certificate distribution and revocation are not implemented.

The signature stage also has a different purpose from public-key encryption. Alice signs with her private RSA key and Bob verifies with her public key. Public-key encryption would use Bob's encryption public key so that only his private key could recover the encrypted information. Here, RSA does not encrypt the file. AES protects the contents, and DH supplies the shared material used to derive its key. E04 contains a 256-byte public signature over the encrypted envelope and tag. It is not a separately stored signature over the plaintext file.

There are limits to the design that the successful tests do not remove. Bob's DH key is static. If that private key is later stolen, an attacker who saved old envelopes can recompute their shared secrets using the recorded Alice public values. The fresh Alice keys do not provide forward secrecy against compromise of Bob's static key. A valid old envelope can also be replayed because there is no persistent record of message identifiers. Refusing to overwrite an existing output file helps prevent accidental data loss, but it is not replay protection.

7 Reflection and recommendation

For Northbridge's prototype, the main recommendation is to keep the separation between encryption, integrity checks and sender authentication. The results make their roles clear. E02 shows the difference in padding and ciphertext length, while T10 and T11 show why encrypted data still needs authentication. Keeping both CBC and CTR is useful for this assignment because their behaviour can be compared using the same inputs. For an operational service, a reviewed protocol using authenticated encryption would reduce the number of details the application has to get right.

Signatures should remain if Northbridge needs to verify who sent a file independently of the shared MAC key. Public keys would need a managed distribution and replacement process. Bob's static DH key should be replaced with an authenticated ephemeral exchange if forward secrecy is required. Message identifiers and a replay policy would also be needed. These changes add key-management work, but they address limitations that cannot be fixed simply by passing more round-trip tests.

One issue encountered during implementation was an incorrect assumption about the misuse messages. They were intended to have equal lengths, but differed by one byte. An assertion caught this before the experiment results were saved. Checking the byte counts identified the problem, and adding the missing byte made both messages 39 bytes long. The corrected run then recovered the complete second message in T19. This was a useful reminder to check experimental inputs instead of relying on how similar two strings look.

Another issue was the library's deprecation warning for finite-field DH. The pinned version still supports the assignment demonstration, and the warning is kept in the log rather than hidden. A maintained system should move to a supported ephemeral ECDH protocol. Private-key encryption also introduces a practical trade-off: passwords help protect stored keys, but users must manage and recover them. Separate key stores would be preferable to the shared demonstration directory. Overall, the prototype demonstrates the required mechanisms, while further engineering would be needed before using it for real sensitive transfers.

Evidence and reproducibility

E01 records the environment and random test-file fingerprint. E02 contains mode parameters, lengths, ciphertexts and tags. E03 records public DH values and key-agreement checks. E04 contains the signed envelope. E05 contains all 25 test records. E06 contains the misuse experiment, and E07 compares an unkeyed hash with HMAC. These entries are stored in evidence/results.json. E08_cli.json contains the separate command-line checks. The README gives the exact commands for installation, testing and file transfer. The environment, timestamp and test-file fingerprint for the recorded run are included in E01.

References

ICS 344 Applied Cryptography assignment brief. RFC 3526 section 3, https://www.rfc-editor.org/info/rfc3526/ . PyCA cryptography 50.0.2 documentation: symmetric encryption, https://cryptography.io/en/50.0.2/hazmat/primitives/symmetric-encryption/ ; Diffie-Hellman, https://cryptography.io/en/50.0.2/hazmat/primitives/asymmetric/dh/ ; RSA, https://cryptography.io/en/50.0.2/hazmat/primitives/asymmetric/rsa/ .